

Question 1: Software Architecture Design

Based on the business requirements, the most suitable architecture for this ticketing system is a Microservices Architecture managed by an API Gateway. This approach directly addresses your need for functional separation, independent scaling, and a single entry point.

1. Architectural Analysis

- **Entry Point (API Gateway):** Acts as the single point of entry to route requests, handle authentication, and provide a security layer (protection against invalid access).
- **Load Balancing:** Distributed workloads are handled by a Load Balancer situated before the services to manage high traffic peaks.
- **Separation of Concerns:** Each functional area (Booking, Payment, etc.) is built as an independent service.
- **Database per Service:** To ensure that maintaining one part doesn't interrupt others, each service has its own dedicated database.
- **Observability:** A monitoring and logging layer tracks system health.

2. High-Level Block Diagram Design

The diagram below illustrates the interaction between user groups, the processing layers, and the isolated data storage.

A. User Groups

- **Customers:** Access via Mobile App (Browse, Book, Pay).
- **Internal Staff/Managers:** Access via Desktop System (Event Management, Sales Tracking).

B. Main Functional Components (Microservices)

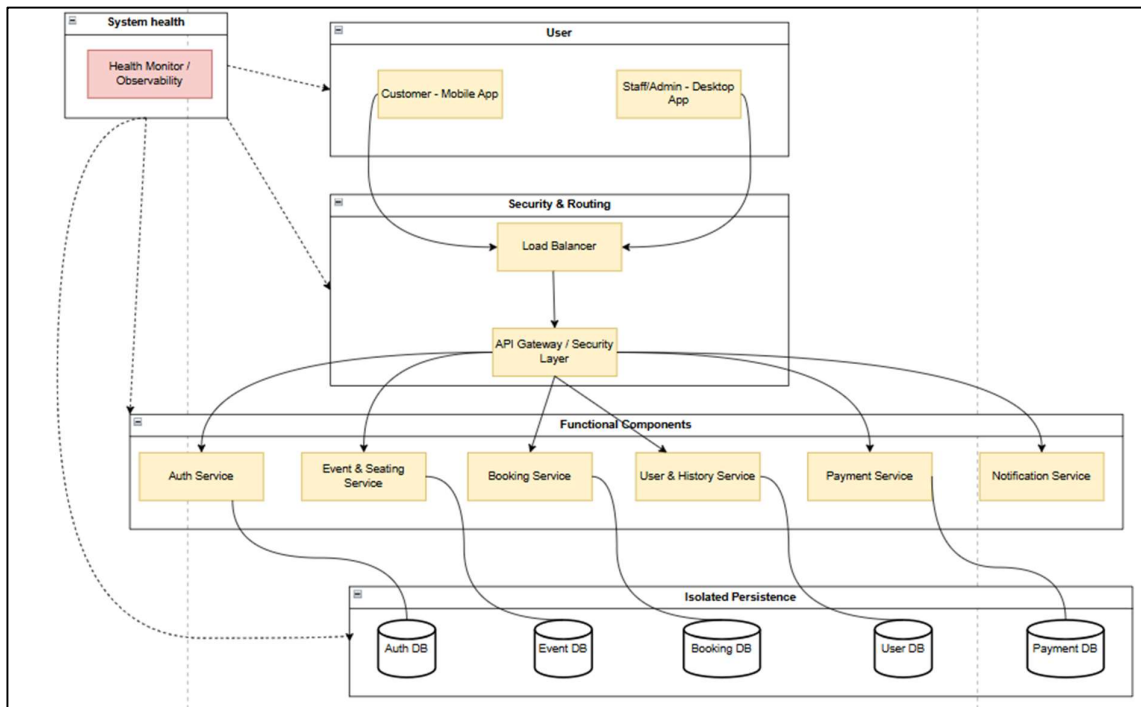
1. **Auth Service:** Handles authentication, authorization, and access control.
2. **Event & Seating Service:** Manages event details and seating layouts.
3. **Booking Service:** Processes ticket reservations and availability.
4. **User Service:** Manages customer profiles and purchase history.
5. **Payment Service:** Integrates with external payment gateways.
6. **Notification Service:** Dispatches Emails/SMS/Push notifications.

C. Data Storage

- **Decoupled Databases:** Each service listed above has its own schema (e.g., EventDB, UserDB, BookingDB) to prevent cascading failures during maintenance.

D. System Management Layer

- **Health Monitor:** A centralized dashboard (like Prometheus/Grafana) to evaluate stability and detect issues.



3. Processing Flow (Buying a Ticket)

1. Request: Customer sends a purchase request from the Mobile App.
2. Gateway: The API Gateway validates the session via the Auth Service and routes the request.
3. Processing: The Booking Service checks seat availability and locks the seat.
4. Payment: The Payment Service processes the transaction.
5. Persistence: The User Service updates the purchase history.
6. Notification: Once successful, the Notification Service is triggered to send a confirmation receipt.

Question 2: (Clean Code & Refactoring)

Given the following code:

```
public class SystemManager
{
    public void DoEverything(string type, double money)
    {
        if(type == "card")
        {
            Console.WriteLine("Pay with card: " + money);
        }
        else if(type == "paypal")
        {
```

```

    {
        Console.WriteLine("Pay with paypal: " + money);
    }
    if(money > 500)
        Console.WriteLine("Send VIP email");
    else
        Console.WriteLine("Send normal email");
    }
}

```

1. Identify Code Smells

In the provided SystemManager class, we can identify several issues:

- Violates Single Responsibility Principle (SRP): The method DoEverything is aptly named because it does too much. It handles both payment logic and notification logic.
- Magic Strings: The strings “card”, “paypal”, and “VIP email” are hardcoded. This makes the code prone to typos and difficult to maintain.
- Poor Naming: SystemManager and DoEverything are “God Object” names-they are too vague and don't describe the specific business purpose. money is also less descriptive than amount.
- Tight Coupling / Low Extensibility: Adding a new payment method (like “Momo”) would require modifying the if-else chain, violating the Open/Closed Principle.

2. Refactored Code

```

public enum PaymentType
{
    Card,
    PayPal
}

public class PaymentProcessor
{
    private const double VipThreshold = 500.0;

    public void ProcessTransaction(PaymentType type, double amount)
    {
        ExecutePayment(type, amount);
        SendCustomerNotification(amount);
    }
}

```

```

private void ExecutePayment(PaymentType type, double amount)
{
    // Removed magic strings by using Enum
    string paymentMethod = type.ToString().ToLower();
    Console.WriteLine($"Pay with {paymentMethod}: {amount}");
}

private void SendCustomerNotification(double amount)
{
    // Encapsulated logic for notification types
    if (amount > VipThreshold)
        Console.WriteLine("Send VIP email");
    else
        Console.WriteLine("Send normal email");
}
}

```

Other version:

```

using System;

// 1. Loại bỏ Magic Strings bằng Const hoặc Enum
public static class PaymentMethods {
    public const string Card = "card";
    public const string Paypal = "paypal";
}

// 2. Tách Interface để giảm phụ thuộc (Strategy Pattern)
public interface IPaymentProcessor {
    void Process(double amount);
}

public class CardPayment : IPaymentProcessor {
    public void Process(double amount) => Console.WriteLine($"Pay with card: {amount}");
}

```

```

}

public class PaypalPayment : IPaymentProcessor {
    public void Process(double amount) => Console.WriteLine($"Pay with paypal:
{amount}");
}

// 3. Tách logic gửi Email ra một lớp riêng (SRP)
public class NotificationService {
    private const double VIP_THRESHOLD = 500.0;

    public void SendConfirmation(double amount) {
        if (amount > VIP_THRESHOLD)
            Console.WriteLine("Send VIP email");
        else
            Console.WriteLine("Send normal email");
    }
}

// 4. Lớp điều phối với tên gọi rõ ràng
public class PaymentManager {
    private readonly NotificationService _notificationService = new NotificationService();

    // Thay vì truyền string "type", ta truyền trực tiếp Object xử lý thanh toán
    public void ExecutePayment(IPaymentProcessor paymentProcessor, double amount) {
        paymentProcessor.Process(amount);
        _notificationService.SendConfirmation(amount);
    }
}

```

3. Refactoring Decisions Explanation

- **Meaningful Naming:** Changed `SystemManager` to `PaymentProcessor` and `DoEverything` to `ProcessTransaction`. This tells the next developer exactly what the class and method do.
- **Separated Responsibilities:** I broke the logic into `ExecutePayment` and `SendCustomerNotification`. This makes the code easier to test and modify.
- **Removed Magic Strings:** Replaced string comparisons with a `PaymentType Enum`. This provides compile-time safety (no more typos like "paypall").
- **Reduced Coupling:** While this is a basic refactor, moving toward private helper methods is the first step in decoupling the logic. (In the next question, we will fully decouple using interfaces).
- **Magic Numbers:** Defined `VipThreshold` as a constant. If the business changes the VIP limit to \$1000, we only change it in one place.

Question 3: OOP + SOLID Redesign

To address the requirements of the Open/Closed Principle (OCP) and Dependency Inversion Principle (DIP), we implement the Strategy Pattern. This decouples the high-level processing logic from the specific payment implementations.

1. Class Structure Description

- **IPaymentMethod (Interface):** The abstraction layer. It defines a common contract (`ProcessPayment`) for all payment types. By depending on this interface, the system follows DIP.
- **Concrete Strategies:** Classes like `CardPayment`, `PaypalPayment`, `MomoPayment`, and `ZaloPayPayment`. Each implements the interface with its specific logic.
- **PaymentProcessor (Context):** The high-level module that executes the payment. It receives an `IPaymentMethod` via Constructor Injection, ensuring it remains “closed” to modifications when new payment methods are added (OCP).

2. Implementation Code (C#)

// 1. Abstraction (Supports Dependency Inversion)

```
public interface IPaymentMethod
{
    void ProcessPayment(double amount);
}
```

// 2. Concrete Implementations (Supports Open/Closed Principle)

```
public class CardPayment : IPaymentMethod
```

```

{
    public void ProcessPayment(double amount) =>
        Console.WriteLine($"Processing Card payment: ${amount}");
}

public class PaypalPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
        Console.WriteLine($"Processing PayPal payment: ${amount}");
}

// New methods can be added easily without touching existing logic
public class MomoPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
        Console.WriteLine($"Processing Momo payment: ${amount}");
}

public class ZaloPayPayment : IPaymentMethod
{
    public void ProcessPayment(double amount) =>
        Console.WriteLine($"Processing ZaloPay payment: ${amount}");
}

// 3. High-level Module
public class PaymentService
{
    private readonly IPaymentMethod _paymentMethod;

    // Dependency is injected, not hardcoded
    public PaymentService(IPaymentMethod paymentMethod)
    {
        _paymentMethod = paymentMethod;
    }
}

```

```

    }

    public void Execute(double amount)
    {
        _paymentMethod.ProcessPayment(amount);
    }
}

```

3. Describe the class structure.

1. Class Structure Components

The design consists of three primary layers:

- The Abstraction Layer (IPaymentMethod): An interface that acts as the "Contract." It defines a single method, ProcessPayment(double amount). This is the most critical part because it allows the rest of the system to interact with any payment method without knowing the specific details of how that payment is processed.
- The Strategy Layer (Concrete Implementations): These are the individual classes for each payment type:
 - CardPayment: Implements logic for credit/debit transactions.
 - PaypalPayment: Implements logic for the PayPal API.
 - MomoPayment: Implements logic for Momo QR/Wallet.
 - ZaloPayPayment: Implements logic for ZaloPay integration.

Each of these classes implements the IPaymentMethod interface. Because they are separate classes, they follow the Single Responsibility Principle.

- The Context Layer (PaymentService): This class manages the execution. Instead of creating a specific payment object inside its methods, it receives an IPaymentMethod through its Constructor (Constructor Injection). It simply calls _paymentMethod.ProcessPayment(amount).

2. Relationship Summary

- Dependency Inversion: The PaymentService (High-level) depends on the IPaymentMethod interface (Abstraction), not on the specific payment classes (Low-level).
- Open/Closed: To add Momo or ZaloPay, we create new classes that implement the interface. The PaymentService code remains untouched and "closed" to modification.
- Composition over Inheritance: The PaymentService has a payment method rather than being a payment method.

3. Visual Representation (UML Description)

1. Interface: `IPaymentMethod` (Top level).
2. Realization (Dashed arrows with hollow triangles): Four lines point from `CardPayment`, `PaypalPayment`, `MomoPayment`, and `ZaloPayPayment` up to the `IPaymentMethod` interface.
3. Association (Solid arrow): `PaymentService` has a solid arrow pointing toward `IPaymentMethod`, indicating it holds a reference to the abstraction.